COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 -	**SECTION:**	CPE21S1
DATE PERFORMED:	07-23-26	**DATE SUBMITTED:**	
NAME:	Andrew S. Doloso Jr.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 3.1 Linked Lists

1. PROCEDURE OUTPUT

Modify the given operations used in the singly linked lists to work on the new construct of a doubly linked list. Provide a screenshot of your code and analysis in table 3-4.

main.cpp

HOA3P1 > PROCEDURE > main.cpp > ...

```cpp
#include <iostream>
#include "Double_LL.h"

int main() {
    // 'C' 'P' 'E'

    // head node
    DoubleList<char>* head = new_Node('C');

    // second node
    DoubleList<char>* second = new_Node('P');

    // third node
    DoubleList<char>* third = new_Node('E');

    // link first node with the second node
    head->next = second;
    second->prev = head;

    // link second node with the third node
    second->next = third;
    third->prev = second;

    std::cout << "Initial Traversal" << std::endl;
    DllTraverse(head);

    // Insert 'G' at the head
    std::cout << "Inserting 'G' at Head" << std::endl;
    DllInsertHead('G', head);
    DllTraverse(head);

    // Insert 'G' at the end
    std::cout << "Inserting 'G' at End" << std::endl;
    DllInsertEnd('G', head);
    DllTraverse(head);

    // Insert 'B' inside the list after 'P' (second node)
    std::cout << "General Insertion ('B' after 'P')" << std::endl;
    DllgeneralInsert('B', second);
    DllTraverse(head);

    // Delete a specific node ('J')
    std::cout << "Deleting node 'J'" << std::endl;
    DllDelete('J', head);
    DllTraverse(head);

    // Clean up memory by deleting the entire list
    std::cout << "Deleting The Entire List" << std::endl;
    DllDeleteList(head);

    if (head == nullptr) {
        std::cout << "The list is successfully deleted and head is null" << std::endl;
    }

    return 0;
}
```

Double_LL.h

HOA3P1 > PROCEDURE > C Double_LL.h > ...

```cpp
#ifndef Double_LL_H
#define Double_LL_H

#include <iostream>

template <typename T>
class DoubleList {
    public:
        T data;
        DoubleList<T>* next = nullptr;
        DoubleList<T>* prev = nullptr;
};

//------------------
// Creating a node
//------------------
template <typename T>
DoubleList<T>* new_Node(T newData) {
    // dynamically allocate memory for a new node
    DoubleList<T>* newNode = new DoubleList<T>;

    // store the data in the node
    newNode->data = newData;

    // since this is a new node, it does not point to any other node
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;
}

//------------------
// Traversal
//------------------
template <typename T>
void DllTraverse(DoubleList<T>* currentNode) {
    DoubleList<T>* tail = nullptr;

    // 1. Forward Traversal
    std::cout << "Forward Traversal:\n";
    while (currentNode != nullptr) {
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
    std::cout << "\n"; // Newline so Reverse Traversal starts on a new line

    // 2. Reverse Traversal
    std::cout << "Reverse Traversal:\n";
    while (tail != nullptr) {
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
    std::cout << "\n\n"; // Newlines to space out calls from main()
}

```

HOA3P1 > PROCEDURE > C Double_LL.h > ...

```
1   #ifndef Double_LL_H
56  //------------------
57  // Insertion at head
58  //------------------
59  template <typename T>
60  void DllInsertHead(T newData, DoubleList<T>*& currenthead) {
61      DoubleList<T>* newNode = new_Node(newData);
62
63      newNode->next = currenthead;
64      newNode->prev = nullptr;
65      if (currenthead != nullptr) {
66          currenthead->prev = newNode;
67      }
68      currenthead = newNode;
69  }
70
71  //------------------
72  // Insertion at the end
73  //------------------
74  template <typename T>
75  void DllInsertEnd(T newData, DoubleList<T>*& currenthead) {
76      // create a new node
77      DoubleList<T>* newNode = new_Node(newData);
78
79      // If the list is empty, new node becomes the head
80      if (currenthead == nullptr) {
81          currenthead = newNode;
82          return;
83      }
84
85      // traverse using a temporary pointer until the last node
86      DoubleList<T>* tail = currenthead;
87      while (tail->next != nullptr) {
88          tail = tail->next;
89      }
90
91      // connect the last node to the new node
92      tail->next = newNode;
93
94      // connect the prev node to the new node
95      newNode->prev = tail;
96  }
97
98  //------------------
99  // General Insertion
100 //------------------
101 template <typename T>
102 void DllgeneralInsert(T newData, DoubleList<T>* prevNode) {
103     if (prevNode == nullptr) {
104         std::cout << "Previous node cannot be null.\n";
105         return;
106     }
107
108     DoubleList<T>* newNode = new_Node(newData);
109
```

HOA3P1 > PROCEDURE > C Double_LL.h > ...

```
1   #ifndef Double_LL_H
102 void DllgeneralInsert(T newData, DoubleList<T>* prevNode) {
110     // Link the new node
111     newNode->next = prevNode->next;
112     newNode->prev = prevNode;
113
114     // If there is a node after prevNode, fix its prev pointer
115     if (prevNode->next != nullptr)
116         prevNode->next->prev = newNode;
117
118     // Finish the insertion
119     prevNode->next = newNode;
120 }
121
122 //-------------------
123 // Deletion
124 //-------------------
125
126 // Deletes a single node by value
127 template <typename T>
128 void DllDelete(T findData, DoubleList<T>*& head) {
129     if (head == nullptr)
130         return;
131
132     DoubleList<T>* currNode = head;
133
134     // Search for the node containing findData
135     while (currNode != nullptr && currNode->data != findData)
136         currNode = currNode->next;
137
138     // Node not found
139     if (currNode == nullptr)
140         return;
141
142     // Update previous node's next pointer or move head pointer
143     if (currNode->prev != nullptr)
144         currNode->prev->next = currNode->next;
145     else
146         head = currNode->next;
147
148     // Update next node's prev pointer
149     if (currNode->next != nullptr)
150         currNode->next->prev = currNode->prev;
151
152     delete currNode;
153 }
154
155 // Deletes the entire list to free memory
156 template <typename T>
157 void DllDeleteList(DoubleList<T>*& head) {
158     DoubleList<T>* currNode = head;
159
160     while (currNode != nullptr) {
161         DoubleList<T>* temp = currNode;
162         currNode = currNode->next;
163         delete temp;
164     }
165     head = nullptr;
166 }
167
168 #endif
```

Output

```
PS C:\Users\Doloso\CPE010-HOA3.1_DOLOSO> cd "c:\Users\Doloso\CPE010-HOA3.
Initial Traversal
Forward Traversal:
C P E
Reverse Traversal:
E P C

Inserting 'G' at Head
Forward Traversal:
G C P E
Reverse Traversal:
E P C G

Inserting 'G' at End
Forward Traversal:
G C P E G
Reverse Traversal:
G E P C G

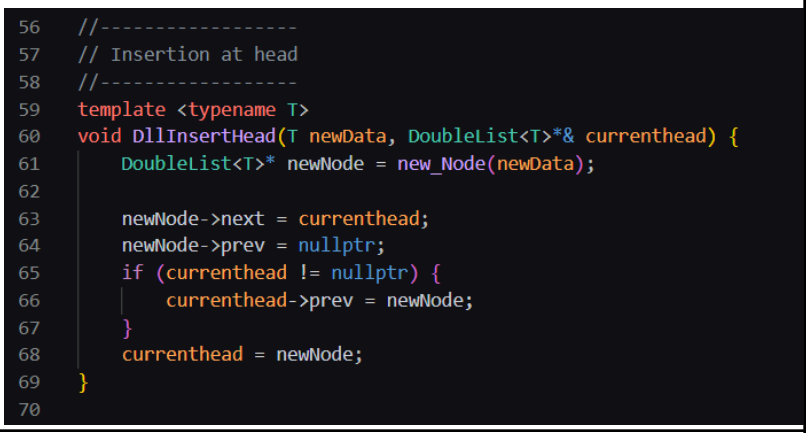
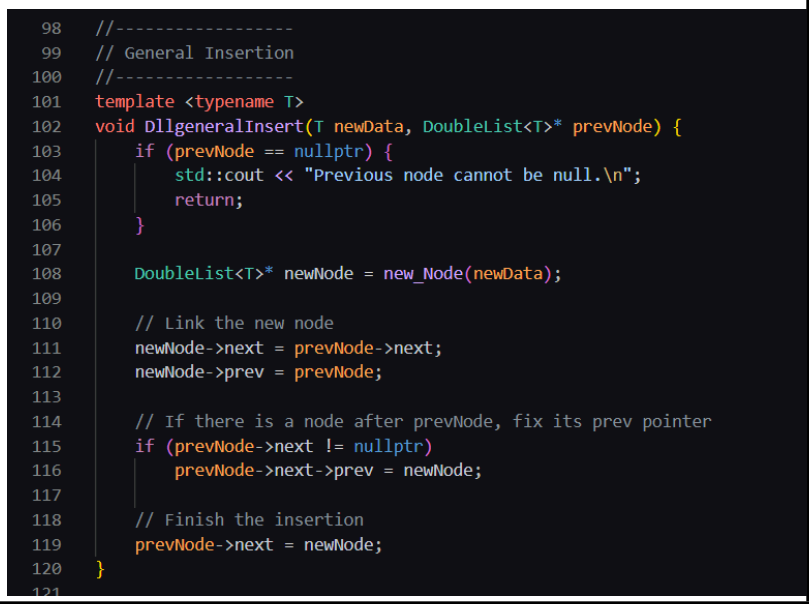
General Insertion ('B' after 'P')
Forward Traversal:
G C P B E G
Reverse Traversal:
G E B P C G

Deleting node 'J'
Forward Traversal:
G C P B E G
Reverse Traversal:
G E B P C G

Deleting Entire List
List successfully deleted and head is null
PS C:\Users\Doloso\CPE010-HOA3.1_DOLOSO\HOA3P1\PROCEDURE>
```

DISCUSSION/LEARNINGS

T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

Operation	Screenshot
Traversal	
Insert at Head	
Insertion at any part	

```cpp
//------------------
// Traversal
//------------------
template <typename T>
void DllTraverse(DoubleList<T>* currentNode) {
    DoubleList<T>* tail = nullptr;

    // 1. Forward Traversal
    std::cout << "Forward Traversal:\n";
    while (currentNode != nullptr) {
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
    std::cout << "\n"; // Newline so Reverse Traversal starts on a new line

    // 2. Reverse Traversal
    std::cout << "Reverse Traversal:\n";
    while (tail != nullptr) {
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
    std::cout << "\n\n"; // Newlines to space out calls from main()
}
```

```cpp
//------------------
// Insertion at head
//------------------
template <typename T>
void DllInsertHead(T newData, DoubleList<T>*& currenthead) {
    DoubleList<T>* newNode = new_Node(newData);

    newNode->next = currenthead;
    newNode->prev = nullptr;
    if (currenthead != nullptr) {
        currenthead->prev = newNode;
    }
    currenthead = newNode;
}
```

```cpp
//------------------
// General Insertion
//------------------
template <typename T>
void DllgeneralInsert(T newData, DoubleList<T>* prevNode) {
    if (prevNode == nullptr) {
        std::cout << "Previous node cannot be null.\n";
        return;
    }

    DoubleList<T>* newNode = new_Node(newData);

    // Link the new node
    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    // If there is a node after prevNode, fix its prev pointer
    if (prevNode->next != nullptr)
        prevNode->next->prev = newNode;

    // Finish the insertion
    prevNode->next = newNode;
}
```

Insertion at the end	

```cpp
//------------------
// Insertion at the end
//------------------
template <typename T>
void DllInsertEnd(T newData, DoubleList<T>*& currenthead) {
    // create a new node
    DoubleList<T>* newNode = new_Node(newData);

    // If the list is empty, new node becomes the head
    if (currenthead == nullptr) {
        currenthead = newNode;
        return;
    }

    // traverse using a temporary pointer until the last node
    DoubleList<T>* tail = currenthead;
    while (tail->next != nullptr) {
        tail = tail->next;
    }

    // connect the last node to the new node
    tail->next = newNode;

    // connect the prev node to the new node
    newNode->prev = tail;
}
```

Deletion of Node

```cpp
// Deletes a single node by value
template <typename T>
void DllDelete(T findData, DoubleList<T>*& head) {
    if (head == nullptr)
        return;

    DoubleList<T>* currNode = head;

    // Search for the node containing findData
    while (currNode != nullptr && currNode->data != findData)
        currNode = currNode->next;

    // Node not found
    if (currNode == nullptr)
        return;

    // Update previous node's next pointer or move head pointer
    if (currNode->prev != nullptr)
        currNode->prev->next = currNode->next;
    else
        head = currNode->next;

    // Update next node's prev pointer
    if (currNode->next != nullptr)
        currNode->next->prev = currNode->prev;

    delete currNode;
}
```

```cpp
// Deletes the entire list to free memory
template <typename T>
void DllDeleteList(DoubleList<T>*& head) {
    DoubleList<T>* currNode = head;

    while (currNode != nullptr) {
        DoubleList<T>* temp = currNode;
        currNode = currNode->next;
        delete temp;
    }
    head = nullptr;
}

#endif
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++
Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing
Problem Description:

In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple Players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported: Create a playlist using multiple songs.
24
Add songs to the playlist.
Remove a song from the playlist.
Play songs in a loop (for this activity, we will print all the songs once). Here are the steps to solve the problem:
Design the basic structure that supports circular data representation. After that, implement the insert and delete functions in the structure. Implement a function for traversing the playlist.
The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

main.cpp

```
#include <iostream>

#include <string>

#include "Double_LL.h"



int main() {

    SingleSongNode<std::string>* playlist = nullptr;

    //added songs to the playlist

    std::cout << "-----My Playlist-----\n";

    addSong(std::string("pag-ibig"), playlist);

    addSong(std::string("buko"), playlist);

    addSong(std::string("teka lang"), playlist);

    addSong(std::string("multo"), playlist);

    addSong(std::string("Simoy"), playlist);



    std::cout << "\n------------------------\n";

    NextSong(playlist);

    PrevSong(playlist);

    std::cout << "------------------------\n";



    std::cout << "\n       Play all \n";

    PlayAll(playlist);



    std::cout << "\n------------------------\n";
```

```
    DeleteSong<std::string>("buko", playlist);

    DeleteSong<std::string>("buko", playlist);

    std::cout << "------------------------\n";



    std::cout << "\n       Play all\n";

    PlayAll(playlist);



    return 0;

}
```

Double_LL.h

```
#ifndef DOUBLE_LL_H

#define DOUBLE_LL_H



#include <iostream>



template <typename T>

class SingleSongNode {

public:

    T SongTitle;

    SingleSongNode<T>* next;

    SingleSongNode<T>* prev;



    SingleSongNode(T title) {

        SongTitle = title;
```

```
        next = nullptr;
        prev = nullptr;
    }
};

// Playlist and adding songs
template <typename T>
void addSong(T newTitle, SingleSongNode<T>*& PlayListHead) {
    static int songCount = 0;
    songCount++;

    SingleSongNode<T>* newNode = new SingleSongNode<T>(newTitle);

    // If the playlist is empty, the new song points to itself
    if (PlayListHead == nullptr) {
        PlayListHead = newNode;
        PlayListHead->next = PlayListHead;
        PlayListHead->prev = PlayListHead;
        std::cout << "\n+-+-+ Added song +-+-+\n";
    } else {
        // Insert at end (circular)
        SingleSongNode<T>* tail = PlayListHead->prev;

        tail->next = newNode;
```

```
        newNode->prev = tail;



        newNode->next = PlayListHead;

        PlayListHead->prev = newNode;

    }



    std::cout << "song " << songCount << ". " << newTitle << "\n";

    std::cout << "------------------------\n";

}



// Next Song

template <typename T>

void NextSong(SingleSongNode<T>*& PlayListHead) {

    if (PlayListHead == nullptr) {

        std::cout << "Playlist is empty.\n";

        return;

    }

    PlayListHead = PlayListHead->next;

    std::cout << "Now playing: " << PlayListHead->SongTitle << "\n";

}



// Prev Song

template <typename T>

void PrevSong(SingleSongNode<T>*& PlayListHead) {
```

```
    if (PlayListHead == nullptr) {
        std::cout << "Playlist is empty.\n";
        return;
    }
    PlayListHead = PlayListHead->prev;
    std::cout << "Previous song: " << PlayListHead->SongTitle << "\n";
}

// Play All
template <typename T>
void PlayAll(SingleSongNode<T>*& PlayListHead) {
    if (PlayListHead == nullptr) {
        std::cout << "Playlist is empty.\n";
        return;
    }

    SingleSongNode<T>* current = PlayListHead;

    do {
        std::cout << "Now playing: " << current->SongTitle << "\n";
        current = current->next;
    } while (current != PlayListHead);
}
```

```cpp
// Delete Song

template <typename T>

void DeleteSong(T title, SingleSongNode<T>*& PlayListHead) {



    if (PlayListHead == nullptr) {

        std::cout << "Playlist is empty.\n";

        return;

    }



    SingleSongNode<T>* current = PlayListHead;



    do {

        if (current->SongTitle == title) {



            // If only one node exists

            if (current->next == current) {

                PlayListHead = nullptr;

            } else {

                // Removing node from multi-node list

                current->prev->next = current->next;

                current->next->prev = current->prev;



                // Deleting head node
```

```
                if (current == PlayListHead) {
                    PlayListHead = current->next;
                }
            }

            delete current;
            std::cout << "Remove from playlist: " << title << "\n";
            return;
        }

        current = current->next;

    } while (current != PlayListHead);

    std::cout << "this song is not in the playlist: " << title << "\n";
}

#endi
```

Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Doloso\CPE010-HOA3.1_DOLOSO\HOA3P1_SPPLMTRY> cd "c:\Users\Doloso\CPE010-HOA
-----My Playlist-----

+-+-+ Added song +-+-+
song 1. pag-ibig
------------------------
song 2. buko
------------------------
song 3. teka lang
------------------------
song 4. multo
------------------------
song 5. Simoy
------------------------

------------------------
Now playing: buko
Previous song: pag-ibig
------------------------

       Play all
Now playing: pag-ibig
Now playing: buko
Now playing: teka lang
Now playing: multo
Now playing: Simoy

------------------------
Remove from playlist: buko
this song is not in the playlist: buko
------------------------

       Play all
Now playing: pag-ibig
Now playing: teka lang
Now playing: multo
Now playing: Simoy
PS C:\Users\Doloso\CPE010-HOA3.1_DOLOSO\HOA3P1_SPPLMTRY>
```

Discussion / Conlusion:

In this activity i understand how a double linked list works through the use of nodes and pointers.
In order for the playlist to function properly, it is necessary to use operations such as navigating the next and previous songs. Because of this lesson,double linked list allows continuous traversal since the last node connects back to the first node, and I also realized that proper pointer handling is important to maintain the structure and avoid errors.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Explaining the error, *grammar correction*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines